

DSA Practical No: 05

Name: More Sahil Sandip PRN: B26ET2022 Div: SY3 Class: C3

Queue Using Array :

INPUT :

```
#include <stdio.h>

#include <stdlib.h>

#define SIZE 5

struct Queue
{
    int q[SIZE];
    int front;
    int rear;
};

int isFull(struct Queue *qu)
{
    if (qu->rear >= SIZE - 1)
        return 1;
    else
        return 0;
}

int isEmpty(struct Queue *qu)
{
    if (qu->front == -1 || qu->front > qu->rear)
        return 1;
```

```
    else
        return 0;
}
```

```
void insert(struct Queue *qu, int item)
```

```
{
    if (qu->front == -1)
        qu->front = 0;

    qu->rear++;
    qu->q[qu->rear] = item;
}
```

```
void delete(struct Queue *qu)
```

```
{
    int item;

    if (isEmpty(qu))
    {
        printf("\nQueue is empty! Underflow.\n");
    }
    else
    {
        item = qu->q[qu->front];

        printf("\nThe deleted element is: %d\n", item);

        qu->front++;
    }
}
```

```
    if (qu->front > qu->rear)
    {
        qu->front = -1;
        qu->rear = -1;
    }
}
```

```
void display(struct Queue *qu)
{
    int i;

    if (isEmpty(qu))
    {
        printf("\nQueue is empty!\n");
    }
    else
    {
        printf("\nQueue elements are:\n");

        for (i = qu->front; i <= qu->rear; i++)
        {
            printf("%d ", qu->q[i]);
        }

        printf("\nFRONT = %d", qu->front);
        printf("\nREAR = %d\n", qu->rear);
    }
}
```

```

    }
}

int main()
{
    struct Queue qu;

    int choice;
    int item;
    char ans;

    qu.front = -1;
    qu.rear = -1;

    do
    {
        printf("IMPLEMENTATION OF QUEUE\n");

        printf("1. INSERT\n");
        printf("2. DELETE\n");
        printf("3. DISPLAY\n");
        printf("4. EXIT\n");

        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {

```

case 1:

```
printf("\nEnter the item to be inserted: ");  
scanf("%d", &item);  
  
if (isFull(&qu))  
{  
    printf("\nQueue is full! Overflow.\n");  
}  
else  
{  
    insert(&qu, item);  
    printf("\nItem %d inserted successfully.\n", item);  
}  
  
break;
```

case 2:

```
delete(&qu);  
  
break;
```

case 3:

```
display(&qu);  
  
break;
```

case 4:

```
printf("\nExiting program...\n");  
exit(0);
```

default:

```
printf("\nInvalid choice!\n");  
}
```

```
printf("\nDo you want to continue? (y/n): ");  
scanf(" %c", &ans);
```

```
} while (ans == 'y' || ans == 'Y');
```

```
printf("\nProgram ended.\n");
```

```
return 0;  
}
```

OUTPUT :

```
PS D:\dsa\Experiment-05-Stack-and-Queue> .\queue.exe
```

```
Do you want to continue? (y/n): y
```

```
IMPLEMENTATION OF QUEUE
```

1. INSERT
2. DELETE
3. DISPLAY
4. EXIT

```
Enter your choice: 1
```

```
Enter the item to be inserted: 14
```

```
Item 14 inserted successfully.
```

```
Do you want to continue? (y/n): y
```

```
IMPLEMENTATION OF QUEUE
```

1. INSERT
2. DELETE
3. DISPLAY
4. EXIT

```
Enter your choice: 1
```

```
Enter the item to be inserted: 15
```

```
Item 15 inserted successfully.
```

```
Do you want to continue? (y/n): y
```

IMPLEMENTATION OF QUEUE

1. INSERT
2. DELETE
3. DISPLAY
4. EXIT

Enter your choice: 1

Enter the item to be inserted: 14

Item 14 inserted successfully.

Do you want to continue? (y/n): y

IMPLEMENTATION OF QUEUE

1. INSERT
2. DELETE
3. DISPLAY
4. EXIT

Enter your choice: 1

Enter the item to be inserted: 15

Item 15 inserted successfully.

Do you want to continue? (y/n): y

IMPLEMENTATION OF QUEUE

1. INSERT
2. DELETE
3. DISPLAY
4. EXIT

Enter your choice: 2

The deleted element is: 12

Do you want to continue? (y/n): y

IMPLEMENTATION OF QUEUE

1. INSERT
2. DELETE
3. DISPLAY
4. EXIT

Enter your choice: 3

Queue elements are:

13 14 15

FRONT = 1

REAR = 3

Do you want to continue? (y/n): y

IMPLEMENTATION OF QUEUE

1. INSERT
2. DELETE
3. DISPLAY
4. EXIT

Enter your choice: 4

Exiting program...

PS D:\dsa\Experiment-05-Stack-and-Queue>